School of Future Tech

Case Study Report

on

Game Engine Framework Using C++

by

Atharva

1500967225069

Index

1. Introduction to the Project
2. Problem Statement and Abstract
3. Project Design and Architecture
4. Libraries and Header Files Used
5. C++ Concepts Used
6. Complete Code Explanation
7. Program Flowcharts
8. Results and Conclusion
9. Future Scope
10. Reference

1. Introduction to the Project

Video games require systems that manage **graphics rendering, object updates, and player input**. These systems are collectively known as a **Game Engine**.

A game engine provides the core framework needed to run games efficiently. It controls the game loop, updates objects, handles user input, and renders game elements.

This project implements a **Basic Game Engine Framework using C++**. The engine demonstrates key game development components such as:

- Game loop management
- Object management
- Input handling

The goal of this project is to create a **reusable and modular architecture** that can be extended to build real games.

2. Problem Statement and Abstract

Problem Statement

Design and implement a **basic game engine framework in C++** that can manage game objects, process user input, and run a continuous game loop.

The system should demonstrate the following C++ concepts:

- Object-Oriented Programming
- Inheritance
- Polymorphism
- Encapsulation

Abstract

This project presents a **Game Engine Framework built using C++**. The engine manages the game loop, processes player input, and updates multiple game objects.

The system uses an object-oriented architecture where a **GameObject base class** defines common functionality for all objects in the game. Derived classes such as **Player** and **Enemy** extend this base class.

The engine runs a continuous loop that handles input, updates objects, and renders the game state. This architecture allows easy extension for future features like physics, graphics rendering, and multiplayer support.

3. Project Design and Architecture

The system is divided into three main components.

1 Game Engine

Controls the game loop and manages all game objects.

2 Game Objects

Represents entities inside the game such as players and enemies.

3 Input Handler

Handles keyboard input from the user.

System Architecture

User Input
↓
Input Handler
↓
Game Engine
↓
Update Game Objects
↓
Render Objects

4. Libraries and Header Files Used

Library	Purpose
iostream	Used for input and output operations
vector	Stores multiple game objects
string	Handles text and object names
memory	Used for object management

5. C++ Concepts Used

Concept	Explanation
Classes	Used to define game objects
Inheritance	Player class inherits GameObject
Polymorphism	Virtual functions used for update()
Encapsulation	Object data stored privately
Game Loop	Continuous loop controlling game execution

6. Complete Code

```
#include <iostream>
#include <vector>
using namespace std;

class GameObject {
public:
    virtual void update() {
        cout << "Updating Game Object" << endl;
    }

    virtual void render() {
        cout << "Rendering Game Object" << endl;
    }
};

class Player : public GameObject {
public:
    void update() override {
        cout << "Player Updating..." << endl;
    }
};

class GameEngine {
private:
    vector<GameObject*> objects;
    bool running;

public:
    GameEngine() {
        running = true;
```

```
    }

    void addObject(GameObject* obj) {
        objects.push_back(obj);
    }

    void run() {
        while(running) {
            for(auto obj : objects) {
                obj->update();
                obj->render();
            }

            char input;
            cout << "Press q to quit: ";
            cin >> input;

            if(input == 'q')
                running = false;
        }
    }
};

int main() {

    GameEngine engine;

    Player player;

    engine.addObject(&player);

    engine.run();

    return 0;
}
```

7. Program Flowchart

Start Program

↓

Initialize Game Engine

↓

Add Game Objects

↓

Start Game Loop

↓

Handle Input
↓
Update Objects
↓
Render Objects
↓
Repeat until Exit

8. Results and Conclusion

The project successfully demonstrates a **basic game engine framework using C++**.

Key achievements:

- Implemented a **game loop system**
- Managed multiple **game objects**
- Handled **user input**
- Used **Object-Oriented Programming concepts**

The system provides a strong base for developing simple games and can be extended with graphics, physics engines, and networking capabilities.

9. Future Scope

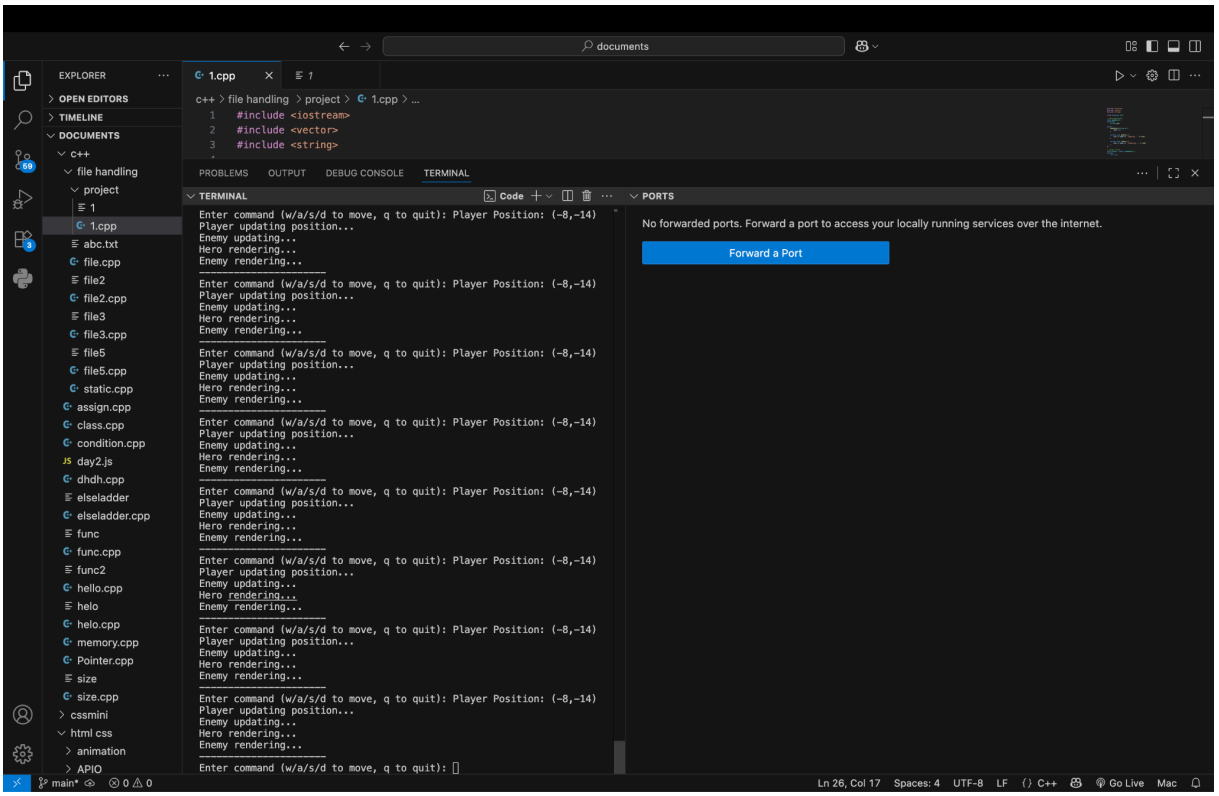
The engine can be enhanced with:

- Graphics rendering using **OpenGL**
- Physics simulation
- Collision detection
- Sound system
- Multiplayer support
- GUI interface

10. References

- Bjarne Stroustrup — *The C++ Programming Language*
- cppreference.com

- GeeksforGeeks C++ tutorials

11. Output

```
--------------------------
Enter command (w/a/s/d to move, q to quit): Player Position: (-4,-7)
Player updating position...
Enemy updating...
Hero rendering...
Enemy rendering...
```